

Secure Software Engineering - 20CYS401

Refactoring Techniques

What Is Refactoring?

- Refactoring is a controlled way of improving the internal structure of existing code without changing what the program actually does from the outside.
- It works by applying a series of small, safe changes, one at a time, rather than one large rewrite. Each change alone may look too small to matter, but together they add up to a significant improvement.
- Because each step is small and behavior stays the same, the system is never left broken while the restructuring happens, and the risk of introducing new bugs stays low.

What Is a Code Smell?

- A code smell is a warning sign in the code — not a bug, and not something that breaks the program, but a pattern that hints a deeper design problem may exist underneath.
- The term was first coined by Kent Beck and later popularized through Martin Fowler's writing on refactoring. Because it is a matter of judgment, what counts as a smell can vary between developers, languages, and teams.
- Smells matter because they slow future work down and raise the chance of bugs appearing later, even though the code runs correctly today.

What Is Technical Debt?

- Technical debt is a metaphor introduced by Ward Cunningham to describe the cost of choosing a quick, imperfect solution now instead of a better one that would take longer.
- Just like a financial loan, this choice comes with interest: every future change becomes a little slower and a little more expensive until the shortcut is fixed.
- In Cunningham's original idea, taking on debt was a conscious, reasonable trade-off, not a mistake. Problems appear only when debt is ignored and allowed to build up unmanaged over time.

Why Refactoring Is Needed

- Software is read and modified far more often than it is written for the first time, so keeping code easy to understand saves far more time than it costs.
- Refactoring keeps technical debt under control, reduces the chance of hidden bugs, and makes it faster and safer to add new features later.
- It also improves teamwork: code that is simple and consistent is easier for colleagues to review, understand, and build on without introducing mistakes.

Code Smells

Long Method

What it looks like: A function or method has grown so large that it is hard to read in one glance, often mixing several unrelated steps together.

Usual fix: Break it apart using Extract Method, pulling each logical step into its own smaller, well-named function.

```
def process_order(order):  
    total = 0  
    for item in order.items:  
        total += item.price * item.qty  
    if order.customer.is_member:  
        total *= 0.9  
    tax = total * 0.08  
    total += tax  
    print(f"Shipping to {order.customer.address}")  
    print(f"Total: {total}")  
    return total
```

Duplicate Code

What it looks like: The same or very similar block of code appears in more than one place in the program.

Usual fix: Extract the shared logic into a single reusable function so there is only one place to update.

```
def print_invoice(order):  
    total = sum(i.price*i.qty for i in order.items)  
    print(f"Total: {total}")  
  
def print_receipt(order):  
    total = sum(i.price*i.qty for i in order.items)  
    print(f"Total: {total}")
```


Primitive Obsession

What it looks like: Plain values like raw numbers or strings are used to represent things that really deserve their own small class, such as money or a status code.

Usual fix: Wrap the primitive in a dedicated class so the concept has a name and can carry its own rules.

```
ROLE_ADMIN = 1
```

```
def create_user(name, role_code):  
    return {"name": name, "role": role_code}
```

Switch Statements / Complex Conditionals

What it looks like: A long chain of if/elif statements or a switch-like block repeatedly checks the same type or status to decide what to do.

Usual fix: Replace the branching logic with polymorphism, letting each type provide its own behavior.

```
def get_speed(bird_type):  
    if bird_type == "European":  
        return 35  
    elif bird_type == "African":  
        return 40  
    elif bird_type == "Norwegian":  
        return 30
```

Long Parameter List

What it looks like: A function takes so many parameters that it becomes difficult to remember their order or purpose when calling it.

Usual fix: Group the related parameters into a single object and pass that instead.

```
def create_booking(guest_name, guest_email, check_in,  
                  check_out, room_type, num_guests):  
    pass
```

Large Class

What it looks like: A class has grown to cover many unrelated responsibilities, making it hard to understand as a single unit.

Usual fix: Split the class using Extract Class, moving related fields and methods into a new, focused class.

```
class Employee:
    def __init__(self, name, salary, address, phone):
        self.name = name
        self.salary = salary
        self.address = address
        self.phone = phone

    def calculate_pay(self):
        return self.salary

    def print_address_label(self):
        print(self.address)
```

Data Clumps

What it looks like: The same small group of variables (like host, port, username) keeps appearing together across different functions.

Usual fix: Bundle the group into its own small class or object so it travels as one unit.

```
def connect(host, port, username, password):  
    pass  
  
def reconnect(host, port, username, password):  
    pass
```

Divergent Change

What it looks like: A single class has to be edited for many unrelated reasons, because it has taken on too many separate jobs.

Usual fix: Split the class so that each responsibility has its own place to change independently.

```
class Product:
    def find(self): pass
    def display(self): pass
    def order(self): pass
    def calculate_tax(self): pass # unrelated concern
```

Shotgun Surgery

What it looks like: A single small change forces you to edit many different classes scattered across the codebase.

Usual fix: Move the related methods and fields together into one class so future changes stay localized.

```
class OrderValidator:
    def check_amount(self, order): pass

class OrderLogger:
    def log_amount_rule(self): pass

class OrderNotifier:
    def notify_amount_rule_change(self): pass
```

Dead Code

What it looks like: A variable, function, or class is left in the codebase even though nothing in the program actually uses it anymore.

Usual fix: Delete it. Version control keeps the history if it is ever needed again.

```
def calculate_total(price, tax, legacy_discount=None):  
    # legacy_discount is never used anymore  
    return price + tax
```


Refactoring Techniques

Extract Method

How it works: Take a chunk of logic from inside a long function and move it into its own separate, clearly named function.

Why it helps: Makes each piece of logic easy to read on its own and lets you reuse it elsewhere without copying code.

Before

```
def print_owing(orders):  
    print("Owing:")  
    total = 0  
    for o in orders:  
        total += o.amount  
    print(f"Total: {total}")
```

After

```
def calculate_total(orders):  
    return sum(o.amount for o in  
orders)  
  
def print_owing(orders):  
    print("Owing:")  
    print(f"Total:  
{calculate_total(orders)}")
```

Replace Magic Number with Symbolic Constant

How it works: Give an unexplained literal number or string a descriptive name by storing it in a constant.

Why it helps: Anyone reading the code later immediately understands what the value represents.

Before

```
area = radius * radius * 3.14159
```

After

```
PI = 3.14159  
area = radius * radius * PI
```

Replace Nested Conditional with Guard Clauses

How it works: Handle special or edge cases first with early return statements, instead of nesting them deep inside if/else blocks.

Why it helps: The normal, expected path through the function becomes immediately visible at the top level.

Before

```
def get_payout(employee):
    result = 0
    if employee.is_separated:
        if employee.is_retired:
            result = 0
        else:
            result =
separation_pay(employee)
    else:
        result = normal_pay(employee)
    return result
```

After

```
def get_payout(employee):
    if employee.is_separated and
employee.is_retired:
        return 0
    if employee.is_separated:
        return
separation_pay(employee)
    return normal_pay(employee)
```

Decompose Conditional

How it works: Pull the condition itself, and each of its branches, out into separate, well-named helper functions.

Why it helps: The purpose of each branch becomes clear from its name instead of from reading the raw logic.

Before

```
if date.month in (12, 1, 2):  
    charge = quantity * winter_rate  
else:  
    charge = quantity * summer_rate
```

After

```
def is_winter(date):  
    return date.month in (12, 1, 2)  
  
charge = (quantity * winter_rate if  
is_winter(date)  
        else quantity *  
summer_rate)
```

Extract Class

How it works: When one class is doing the work of two, move part of its fields and methods into a brand-new class.

Why it helps: Each resulting class stays focused on a single responsibility, making both easier to test and maintain.

Before

```
class Person:
    def __init__(self, name,
area_code, number):
        self.name = name
        self.area_code = area_code
        self.number = number
```

After

```
class PhoneNumber:
    def __init__(self, area_code,
number):
        self.area_code = area_code
        self.number = number

class Person:
    def __init__(self, name,
area_code, number):
        self.name = name
        self.phone =
PhoneNumber(area_code, number)
```

Inline Method

How it works: If a method's body is just as clear as its name, replace calls to it with the body itself and delete the method.

Why it helps: Removes an extra layer of indirection that was not actually adding any clarity.

Before

```
def more_than_five(driver):  
    return driver.deliveries > 5  
  
def get_rating(driver):  
    return more_than_five(driver)
```

After

```
def get_rating(driver):  
    return driver.deliveries > 5
```

Extract Variable

How it works: Give a piece of a long or confusing expression its own short, descriptive variable name.

Why it helps: Turns a hard-to-parse line of logic into something that reads almost like plain language.

Before

```
if platform.upper().startswith("MAC") and browser.upper().startswith("IE"):
    do_something()
```

After

```
is_mac = platform.upper().startswith("MAC")
is_ie = browser.upper().startswith("IE")
if is_mac and is_ie:
    do_something()
```


Introduce Parameter Object

How it works: Replace a group of parameters that are always passed together with a single object carrying all of them.

Why it helps: Shortens method signatures and keeps naturally related data traveling together as one unit.

Before

```
def get_bookings(start_date, end_date,  
                 guest_name, guest_email):  
    pass
```

After

```
from dataclasses import dataclass  
  
@dataclass  
class BookingRequest:  
    start_date: str  
    end_date: str  
    guest_name: str  
    guest_email: str  
  
def get_bookings(request:  
                 BookingRequest):  
    pass
```

Replace Conditional with Polymorphism

How it works: Move each branch of a type-based conditional into its own subclass method, letting the object decide its own behavior.

Why it helps: Removes repetitive type-checks and makes it safe to add new types without touching existing logic.

Before

```
class Bird:
    def __init__(self, kind):
        self.kind = kind
    def speed(self):
        if self.kind == "European":
            return 35
        elif self.kind == "African":
            return 40
```

After

```
class Bird:
    def speed(self):
        raise NotImplementedError

class EuropeanSwallow(Bird):
    def speed(self):
        return 35

class AfricanSwallow(Bird):
    def speed(self):
        return 40
```

Rename Method

How it works: Change a method's name so that it clearly and honestly describes what it actually does.

Why it helps: Other developers can understand what a call does just by reading its name, without opening the function.

Before

```
def calc(items):  
    return sum(i.price for i in  
items)
```

After

```
def get_total(items):  
    return sum(i.price for i in  
items)
```

References

- <https://refactoring.guru>